

PROCRV

Pre-Requirements Operating Concept Rationale and Validation (PROCRV)

Fork-update Agent

December 10, 2025 • v1.1

This document communicates user needs and proposed system concepts to stakeholders for consensus and validation.

Document Layout

Section 1: Identify the scope of the work within the full system.

Section 2: Reference the supporting documents used to create this document.

Section 3: Review the current system state.

Section 4: Justify the proposed changes.

Section 5: Describe the concept for the proposed changes.

Section 6: Outline example scenarios that illustrate the concept (if needed).

Section 7: Describe impacts of the proposed development effort.

Section 8: Analyze the proposed system.

Contents

1	Scope	4
1.1	Identification	4
1.2	Overview	4
2	Referenced Documents	4
3	Current System or Situation	4
3.1	Background, Objectives, and Scope of the Current System	4
3.2	Policies or Constraints That Apply to the Current System	5
3.3	Description of the Current System	5
3.4	Users and Other Key Individuals	6
3.5	Support Concept	6
4	Justification for and Nature of Changes	6
4.1	Justification	6
4.2	Description of Needed Changes	7
4.3	Priorities Among the Changes	7
4.4	Changes Considered but Not Included	8
5	Concept for New or Modified System	9
5.1	Background, Objectives, and Scope of the New System	9
5.2	Policies or Constraints That Apply to the New System	9
5.3	Description of the New or Modified System	9
5.3.1	Implementation Differences vs. Intended Specification	10
5.4	Users and Affected Individuals	10
5.5	Support Concept	11
6	Scenarios	11
6.1	Scenario 1: Routine Update Detection and Manual Review	11
6.2	Scenario 2: Deployment Failure and Notification	12
6.3	Scenario 3: Manual Trigger for On-Demand Execution	12
7	Impacts	12
7.1	Operational Impacts	12
7.2	Organizational Impacts	13
7.3	Impacts During Development	13
7.3.1	Use of LLMs During Development	13

8	Analysis of the Proposed System	14
8.1	Summary of Advantages	14
8.2	Phase 1 Scope and Implementation	14
8.3	Proof of Concept Validation	15
8.4	Summary of Disadvantages and Limitations	15
8.5	Alternatives and Trade-Offs Considered	15

Section 1 – Scope

1.1 Identification

The Fork-Update Agent project enhances the AWS GenAI IDP accelerator by automating updates to the internal fork of `idp_common`. It operates in the AWS sandbox account, with `bpodev/sandbox` as the non-production target environment. The initiative is sponsored by the Product Owner, Jeremy Jacobson, and serves the Design team, Product team, and IDP Engineering as its primary stakeholders. The purpose of the project is to automate upstream `idp_common` updates, validate changes through tests and an end-to-end smoke run, and present clear, consumable results to stakeholders.

1.2 Overview

The accelerator is already deployed in the sandbox, and the Fork-Update Agent extends it with a CI/CD loop that detects new upstream `idp_common` releases, updates the fork, runs unit tests, deploys changes to the sandbox, triggers a Step Functions smoke execution on a sample document, inspects CloudWatch logs, and reports pass/fail outcomes with links to evidence. The business goal is to reduce manual maintenance and provide predictable, auditable updates for pre-sales demos and internal testing.

Section 2 – Referenced Documents

This project relies on three primary sets of reference materials. The PROCRV standard pages provide the source template and section guidance used to structure this document. The existing GenAI IDP CloudFormation stacks in the sandbox (both root and pattern stacks), along with their Outputs such as the Web UI URL and S3 input/output buckets, define the current deployment topology and integration points. The accelerator/`idp_common` repository documentation, including release notes and deployment steps, describes how the shared library evolves and how it is currently built and deployed into the environment.

Section 3 – Current System or Situation

3.1 Background, Objectives, and Scope of the Current System

The current system centers on the sandbox extraction workflows built from the capture and conversion repositories, which Design and Product teams use to validate extraction and classification quality for prospective customers before contract signature. These workflows run on top of a fork of `idp_common` that must periodically be updated from the upstream repository. The demo portal is an application layer that invokes these sandbox pipelines for pre-sales demos, but it is not part of the production extraction flow. Today, updates to the `idp_common` fork and the associated extraction code are performed manually, which introduces opportunities for configuration drift, regressions, and slow turnaround when stakeholders request newer capabilities.

The primary objective of the current setup is to keep the forked `idp_common`-based extraction workflows reasonably current and validated with minimal disruption, while still giving Design and

Product teams enough stability to run reliable demos. In practice, this objective is only partially met, because manual updates are time-consuming and often deferred. The scope of the current system includes three CloudFormation stacks in the sandbox (IDP-ACCELERATOR-TEST-2, IDP-ACCELERATOR-TEST-2-DOCUMENTBEDROCKKB-18IZNMKCSRNC, and IDP-ACCELERATOR-TEST-2-PATTERN2STACK-OTGJJG1KSPG6), Step Functions workflows orchestrating multiple Lambda functions, document ingress and egress via S3, metadata stored in DynamoDB, and a UI secured with Cognito. All updates and deployments to this topology are executed through manual or semi-manual processes.

3.2 Policies or Constraints That Apply to the Current System

The current system operates under constraints that will continue to apply to the new agent. All changes are confined to the sandbox environment; this project explicitly does not modify production accounts or production traffic. Access is governed by least-privilege IAM policies, with roles assumed from CI or developer workstations using OIDC where applicable. Bedrock models used by the accelerator require pre-approval per account, and existing quotas must be respected when running validation or demo workloads.

Change control is handled through branches, pull requests, and CloudFormation change sets, with deployments gated into bpdev/sandbox. Any documents that may contain PII are handled under existing internal data-handling policies, which emphasize minimal retention and careful use of shared buckets. Finally, the system must stay within service quotas and cost caps for Bedrock, Step Functions, SQS, and related AWS services, which limits how aggressively tests and demos can be exercised.

3.3 Description of the Current System

The current system consists of several major components connected through AWS-managed services. Documents are ingested into S3 buckets, processed through Step Functions workflows that orchestrate Lambda tasks built on top of the `idp_common` fork and the capture and conversion code, and written back to S3 as structured outputs. DynamoDB tables store metadata and execution state, while CloudWatch provides logging and basic observability for the pipelines. A web UI, fronted by Cognito for authentication, sits on top of these services and exposes demo flows to internal stakeholders and customer-facing teams. The entire topology is provisioned through a set of nested CloudFormation stacks in the sandbox.

Externally, the system interfaces with Bedrock for model inference. Its primary capabilities include document ingestion, classification, and field extraction, along with the ability to surface results and execution traces for review. Inputs are primarily TIFF images, with some PDF documents, both supported by `idp_common`. Outputs include JSON or CSV payloads, plus CloudWatch logs and Step Functions execution histories that describe how each run progressed.

From a performance perspective, end-to-end latency for a single document is on the order of minutes, with throughput constrained by Step Functions and Lambda concurrency as well as model rate limits. Quality attributes include basic reliability via Step Functions retries, and reasonable observability through logs and execution graphs. However, maintainability is limited by the manual

nature of fork synchronization and deployment, which makes updates slower and more error-prone. Safety, security, and privacy are addressed through sandbox isolation, IAM boundaries, log retention policies, use of non-production data only, and default non-prod backup configurations.

3.4 Users and Other Key Individuals

The primary users of the current IDP sandbox environment and its manual update workflow are the engineering and support teams responsible for keeping the accelerator available. Developers carry out the merge–build–deploy steps by hand, monitor results, and diagnose regressions when updates go wrong, which ties this work directly to their day-to-day planning. Support engineers and on-call staff rely on the sandbox as a reference environment and are responsible for restoring service when manual changes introduce instability. Design and Product teams depend on sandbox outputs, but they do not directly experience the operational effects of delayed updates; breakages surface indirectly through demo-portal issues rather than as blockers to their day-to-day work. Operations or platform teams may also become involved when IAM permissions, network access, or shared infrastructure require adjustment. Overall, the same stakeholder groups that will depend on the Fork-Update Agent in the future are already tightly coupled to the current manual process, but with higher operational friction and less transparency.

3.5 Support Concept

Support for the current system is also anchored in the engineering team and existing operational practices. The accelerator’s capture and conversion code, the `idp_common` fork, and the CloudFormation definitions live in standard repositories, and updates are applied through a mix of ad hoc commands and existing CI/CD jobs rather than through a dedicated automation workflow. Developers and support staff follow runbooks or informal knowledge to decide when to pull upstream changes, how to perform merges, and how to roll back when deployments fail. In day-to-day operation, the sandbox surfaces health primarily through application logs, CloudWatch metrics, and manual test runs; any issues discovered by Design or Product are reported back to Engineering for investigation and remediation.

Maintenance follows the same three categories that will apply to the new agent: corrective work to fix regressions or broken pipelines, adaptive work to reflect changes in upstream `idp_common` or the sandbox infrastructure, and perfective work to simplify scripts, improve documentation, and tighten monitoring. However, without a unified automation layer, this support model remains more person-dependent and less repeatable than the concept defined for the Fork-Update Agent.

Section 4 — Justification for and Nature of Changes

4.1 Justification

The IDP sandbox environment depends operationally on extraction pipelines (the capture and conversion repositories) that are built on top of a fork of the shared upstream library `idp_common`. Upstream `idp_common` evolves frequently and sometimes introduces changes that impact build behavior, configuration schemas, or runtime execution. Today, keeping our sandbox fork aligned

with upstream is still costly and fragile: developers or support engineers must discover new releases, interpret release notes, sync or merge upstream changes into the fork, rebuild and redeploy the extraction stacks, and then manually verify whether the end-to-end pipelines still work. When problems occur, the effort to diagnose them can range from minutes to hours or even days, depending on the size and complexity of the upstream delta. The most impacted stakeholders are therefore the engineers responsible for keeping the extraction pipelines healthy, because upstream changes can break the pipelines directly and create triage and rework. Design and Product stakeholders are primarily consumers of the demo portal and sandbox outputs; they are not operators of the update process, but they depend on the sandbox remaining stable and current for demonstrations and internal evaluations.

In addition to the time cost, the manual process introduces operational risk. It is easy for updates to produce “silent failures” (for example, subtle configuration mismatches or schema changes that do not immediately crash the system but degrade pipeline correctness), and it is difficult to reconstruct exactly what changed and why if the only evidence is fragmented across pull requests, build logs, CloudWatch logs, and informal notes. In practice, `idp_common` may also introduce breaking changes without a major version bump, which makes “version number” alone an unreliable signal of update risk. These factors justify introducing a Fork-Update Agent that standardizes detection, validation, reporting, and traceability so that upstream changes can be consumed in a repeatable and auditable way.

4.2 Description of Needed Changes

To address these limitations, we require an automated mechanism that owns the key parts of the sandbox update lifecycle and clarifies the boundary between what the agent does and what humans do. At minimum, the system must (1) detect new upstream `idp_common` releases or tags, (2) maintain an explicit record of the currently deployed/approved version in the sandbox, (3) notify engineers with sufficient context to review release notes and approve a sync step, (4) deploy the updated version into `bpdev/sandbox` through infrastructure automation, (5) run lightweight end-to-end validation (a smoke test) on curated sample documents, and (6) report outcomes with links to evidence (Step Functions execution, CloudWatch logs, CloudFormation events). Persisting metadata about each update attempt (upstream version, timestamps, outcomes, and minimal diagnostics) is required so that stakeholders can quickly answer operational questions such as “What is currently deployed?” and “When did it change?”

Importantly, the needed change is not only automation for its own sake; it is standardization. The Fork-Update Agent should encode a consistent workflow for updates so that every upstream change follows the same steps and produces comparable results. In Phase 1, this includes human review and approval of upstream changes before the agent proceeds to deploy and validate, which aligns with credentialing and governance constraints around repository write access.

4.3 Priorities Among the Changes

The work is prioritized toward a reliable minimum viable automation loop that matches what is actually implemented in the repository today. The Phase 1 MVP is built around five Lambda

functions, each aligned to a single operational responsibility: `DetectReleaseFn` (detect upstream changes and compare to the recorded sandbox version), `PrepareMergeFn` (notify engineers and gate the workflow based on human review/approval), `DeploySandboxFn` (deploy the approved version into the sandbox via CloudFormation stack updates), `RunSmokeTestFn` (execute lightweight end-to-end validation using the existing IDP Step Functions workflow and curated sample documents in S3), and `ReportStatusFn` (report success/failure via notifications and update recorded state). This set of five functions provides a complete, observable pipeline from detection through validation and reporting, while keeping security and governance boundaries explicit.

A critical guardrail in this priority set is the smoke test (lightweight validation). The smoke test uses a curated set of representative sample documents (primarily TIFF and sometimes PDF) and a pass/fail mechanism to detect breaking upstream changes quickly. The goal is not to measure extraction accuracy exhaustively in the MVP, but to ensure the sandbox workflow remains functional end-to-end after an update: the Step Functions pipeline must start, process the sample document, and complete successfully without runtime errors. This guardrail reduces the risk of deploying upstream changes that break the extraction pipeline in ways that would otherwise be discovered only during demos or ad hoc usage.

To avoid ambiguity in operational reporting, “tests fail” must be interpreted consistently in the agent context. There are three main failure classes: (1) build-level failures (e.g., CodeBuild/unit tests/linting failures, when such checks are enabled), (2) deployment-level failures (e.g., CloudFormation stack update failures, change set failures, or rollback events), and (3) end-to-end validation failures (e.g., the smoke test Step Functions execution fails, times out, or produces invalid outputs). The agent should report which class occurred so engineers can triage efficiently. Secondary priorities for later iterations include richer regression evaluation on a larger curated dataset, more detailed metrics and dashboards, automated PR creation or fork synchronization using a managed service account, and more advanced deployment strategies (such as canary or blue/green approaches) to reduce risk when updating extraction stacks.

4.4 Changes Considered but Not Included

Several alternative approaches were considered and intentionally not included in the initial scope. One option was to keep the manual process and rely on improved documentation only. This was rejected because it does not address inconsistency, silent failures, or the engineering time cost of repeated manual steps. Another option was to implement fully autonomous repository synchronization (automatic merges and PR creation) as part of the MVP. This was deferred due to credentialing and governance constraints; Phase 1 keeps humans in the loop for upstream review and approval, while still automating detection, deployment, validation, and reporting. A third option was to build a comprehensive evaluation platform in the first phase, including deep extraction-quality scoring and richer analytics. This was deferred to avoid expanding scope beyond what is feasible for a reliable internship-scale MVP; the initial system focuses on workflow health and breakage detection via smoke testing. Finally, long-term systems such as drift detection across configuration schemas or broad regression benchmarking are acknowledged as valuable, but they

are treated as separate future efforts rather than being bundled into the core Fork-Update Agent MVP.

Section 5 – Concept for New or Modified System

5.1 Background, Objectives, and Scope of the New System

The Fork-Update Agent is an automation component that sits between the upstream `idp_common` repository and the `bpodev/sandbox` deployment of the GenAI IDP accelerator. Its main objective is to keep the sandbox environment reasonably current with upstream changes without sacrificing stability, auditability, or requiring constant manual effort from developers. The agent does this by standardizing the update lifecycle: detecting upstream releases, recording the deployed/approved version, notifying engineers for review and approval, deploying updates through infrastructure automation, running lightweight end-to-end validation, and reporting outcomes with evidence. The scope is deliberately limited to the sandbox environment and the extraction stacks that depend on the `idp_common` fork; it does not manage production deployments and does not change production workflows. The demo portal is treated as a consumer of the sandbox environment rather than a system dependency that drives updates; the operational focus is on ensuring the extraction pipelines remain healthy and current.

5.2 Policies or Constraints That Apply to the New System

The Fork-Update Agent must operate within security, governance, and operational constraints that match the sandbox environment. It must use least-privilege IAM roles for reading configuration, invoking builds, updating CloudFormation stacks, and executing Step Functions workflows. It must not touch production accounts or production traffic; all actions are confined to `bpodev/sandbox` and its associated AWS resources. Documents used for validation may be sensitive and must be handled under existing internal data-handling policies (minimal retention, controlled access, and careful logging). The agent must also respect service quotas and cost caps, particularly for Step Functions, Lambda concurrency, and any model-related runtime limits.

In addition, implementation and future evolution should align with internal IDP Guidance documents and Architectural Decision Records (ADRs) that capture approved patterns for infrastructure, credentials, and deployment practices. While the MVP uses AWS CDK for reproducible deployment of the agent stack, future iterations should consider alignment with preferred infrastructure tooling such as Terraform if required by broader platform standards. Repository-write automation must follow governance: personal credentials should be avoided, and service accounts or managed tokens (stored in SSM Parameter Store or a secrets manager) should be used.

5.3 Description of the New or Modified System

At a high level, the Fork-Update Agent is implemented as a stateful workflow orchestrated by AWS Step Functions and backed by a set of small, focused Lambda functions. An EventBridge schedule triggers the workflow periodically (for example, every six hours) to check for upstream updates. `DetectReleaseFn` calls the upstream repository API to identify the latest release or tag

and compares it to the last recorded sandbox version stored in SSM. If no update is required, the workflow can exit cleanly with a “no change” outcome.

When a new upstream version is detected, **PrepareMergeFn** issues a notification to engineers (via SNS) with the version metadata and release notes, making the human approval boundary explicit. In the Phase 1 flow, engineers review the upstream change and perform the fork synchronization in the repository under controlled credentials. After that approval step is complete, the agent proceeds to deployment. **DeploySandboxFn** performs a sandbox deployment by updating CloudFormation stack parameters (for example, a version or artifact parameter that controls which `idp_common` fork version is consumed by the extraction stacks), initiating a stack update, and waiting for CloudFormation to complete. The function records deployment evidence through CloudFormation events and CloudWatch logs.

After deployment, **RunSmokeTestFn** runs lightweight end-to-end validation by invoking the existing IDP Accelerator Step Functions workflow using a curated fixture document stored in S3 (typically TIFF, sometimes PDF). The smoke test is treated as the pass/fail guardrail for the update: the workflow must complete successfully to consider the sandbox healthy. Finally, **ReportStatusFn** consolidates outcomes and publishes clear success/failure notifications with execution identifiers and log references, and updates the recorded sandbox state in SSM (e.g., last known good version). Together, these five Lambdas and the Step Functions orchestration provide a repeatable and observable pipeline that reduces manual maintenance and improves transparency.

5.3.1 Implementation Differences vs. Intended Specification

The initial specification envisioned a fully autonomous merge/build stage that could continuously synchronize the fork from upstream, including automated PR creation and conflict resolution workflows. In the implemented Phase 1 MVP, repository-write automation is intentionally constrained: **PrepareMergeFn** is implemented as a notification and gating function rather than an automated merge engine, because repository credentialing and governance (service accounts, managed PATs, and access controls) must be finalized before safe automation can be enabled. Similarly, the MVP emphasizes workflow health validation (smoke testing) rather than comprehensive extraction-quality regression scoring. Artifact publishing into an internal repository (e.g., Artifactory) and broader synchronization workflows (such as mirroring `idp_common` into Azure DevOps for tighter control, using Azure DevOps service accounts and PATs, and automating synchronization pipelines) are treated as planned extensions rather than current capabilities. The MVP also deploys updates via direct CloudFormation stack updates; more advanced deployment strategies such as canary or blue/green rollouts are identified as future enhancements.

5.4 Users and Affected Individuals

The primary users impacted by the Fork-Update Agent are the developers and support engineers responsible for maintaining the sandbox extraction pipelines. They benefit from consistent detection, standardized deployment steps, and clear evidence when failures occur, reducing the time spent on manual monitoring and ad hoc troubleshooting. On-call and DevOps/support roles benefit from improved observability and more structured failure reporting, which simplifies incident

response and reduces the need to reconstruct update history manually. Design and Product teams are secondary beneficiaries: they depend on the sandbox and demo portal being stable and current for demos and evaluations, but they are not the operators of the update process. Overall, the agent reduces operational friction for technical users while increasing reliability and transparency for stakeholders who consume sandbox outputs.

5.5 Support Concept

The Fork-Update Agent is designed to be supported through standard engineering ownership and existing operational tooling. Its code, infrastructure definitions, and documentation (including runbooks) live in a dedicated repository under version control. The agent emits CloudWatch logs for each Lambda and retains Step Functions execution histories as a primary audit trail. Notifications are delivered through SNS so that stakeholders receive consistent outcomes and can quickly navigate to the relevant execution evidence.

Operationally, the agent supports both scheduled and manual triggers. In addition to the EventBridge schedule, engineers can manually trigger an on-demand run using the AWS CLI. A typical command is:

```
aws stepfunctions start-execution \  
  --state-machine-arn "arn:aws:states:us-east-1:922653976899:stateMachine:ForkUpda  
teStateMachine052A8A50-1FuHXHsbZu8E" \  
  --name "manual-test-$(date +%s)" \  
  --input '{}'
```

This manual trigger method is documented for practical use, but it should be reviewed and improved over time (for example, wrapping it in a small script that validates required parameters, enforces naming conventions, and prevents concurrent runs). Maintenance follows three standard categories: corrective maintenance to fix bugs and integration issues, adaptive maintenance to handle upstream `idp_common` changes and evolving sandbox infrastructure, and perfective maintenance to improve validation coverage, tighten reporting, and refine rollback behavior as operational experience accumulates. Documentation, ADR references, and runbooks are updated alongside code changes so that future developers can operate the system without depending on original authors.

Section 6 — Scenarios

This section presents three scenarios illustrating the Fork-Update Agent's operation, stakeholder interactions, and system interfaces.

6.1 Scenario 1: Routine Update Detection and Manual Review

When a new `idp_common` release is published, the agent detects version drift during its six-hour monitoring cycle and sends an SNS notification to subscribed team members. The notification includes the upstream version number, a direct link to the release notes on GitHub, and instructions for manual fork synchronization. Engineering reviews the changes and manually syncs the fork using GitHub's built-in sync interface when ready.

On the next scheduled monitoring cycle (within 6 hours), the agent detects that the fork has been updated with new code and automatically initiates the deployment workflow. The agent deploys the build to bpodev/sandbox via CloudFormation stack update, then performs a smoke-test validation by invoking the existing IDP Step Functions workflow on a known fixture document to confirm the pipeline can start, run, and complete without runtime errors. Upon successful validation, the agent sends a status notification via SNS and updates the SSM parameter tracking the deployed version. Engineering and Product teams receive consistent, tested builds for development and demonstrations while maintaining manual control over merge decisions.

6.2 Scenario 2: Deployment Failure and Notification

When deployment or validation fails, whether due to CloudFormation stack errors, configuration mismatches, or Step Functions workflow failures, the agent logs detailed diagnostics to CloudWatch and sends a failure notification via SNS. The notification documents the failure type and includes CloudFormation events or Step Functions execution details for troubleshooting. The sandbox remains at the previous working version since deployment failures prevent completion of the stack update.

Engineering (and DevOps) teams receive immediate notification of failures without needing to actively monitor the deployment process. Note that automated rollback is not implemented in Phase 1; CloudFormation's natural failure handling prevents partial deployments, but reverting to previous versions requires manual intervention if needed.

6.3 Scenario 3: Manual Trigger for On-Demand Execution

Engineering can manually trigger the agent through the AWS console when immediate validation is needed rather than waiting for the next scheduled run (up to 6 hours). From the Step Functions console, users initiate execution of the Fork-Update Agent state machine, which executes the full detect–deploy–validate workflow when there are changes to apply. The agent checks for upstream changes, deploys if the fork has been updated, runs smoke tests via Step Functions invocation, and sends status notifications.

This manual trigger capability enables on-demand validation when preparing for demonstrations or when verifying that a recent manual fork sync deployed successfully. Engineering receives evidence of successful deployment and validation without waiting for the next scheduled cycle.

Section 7 – Impacts

7.1 Operational Impacts

The agent transforms upstream version monitoring from manual periodic checking to automated detection on a six-hour schedule. This requires updating operational runbooks to cover SNS subscription setup, notification interpretation, fork sync procedures, manual trigger execution via AWS console, and CloudWatch log review for troubleshooting.

The agent integrates with CloudWatch for centralized logging, providing clear visibility into detection cycles, deployment execution, and validation results. All executions produce CloudWatch

logs that support audit trails and trend analysis. The agent tracks the currently deployed version in SSM Parameter Store, providing a single source of truth for version state.

Update cadence shifts from infrequent manual checks (potentially weeks or months between updates) to continuous automated monitoring with maximum 6-hour detection latency. Engineering maintains control over merge timing through the manual fork sync gate, but gains automated awareness of available updates and automated deployment validation after manual merges.

7.2 Organizational Impacts

Ownership boundaries shift within the RACI framework. Engineering owns and maintains the agent infrastructure and is accountable for SNS subscription management, fork sync decisions, and deployment validation. Product becomes a consumer of automatic update notifications, gaining visibility into available upstream releases without requiring Engineering intervention.

The agent is responsible for executing detection, notification, and post-merge deployment workflows. Engineering is consulted and makes decisions on merge timing. Product and Design can be informed through notification subscriptions, gaining awareness of sandbox version state for demonstration planning.

Collaboration patterns transform from synchronous communication about upstream versions to asynchronous notification-based awareness. Product can subscribe to update notifications to understand sandbox currency without interrupting Engineering workflows. Engineering no longer needs to remember to manually check GitHub for upstream releases.

7.3 Impacts During Development

Development required establishing notification subscription procedures, testing fork sync and deployment workflows, and validating error handling across all Lambda functions. The deployment phase involved CDK bootstrap and stack deployment to the sandbox account, SNS topic subscription setup for team members, and EventBridge schedule configuration for six-hour execution cycles.

Infrastructure work included five Lambda function implementations (DetectReleaseFn, PrepareMergeFn, DeploySandboxFn, RunSmokeTestFn, ReportStatusFn), Step Functions state machine orchestration, CloudWatch log group configuration, IAM roles for Lambda permissions, and SSM Parameter Store setup for version tracking.

Documentation proceeded in parallel with development, producing a PROCRV document defining scope and design decisions, an operational runbook covering deployment and usage procedures, and a comprehensive repository README. All documentation was completed to enable operational handoff without requiring intern availability.

7.3.1 Use of LLMs During Development

Large Language Models (LLMs) were used extensively during the development of the Fork-Update Agent as accelerators for documentation, formatting, code synthesis, and troubleshooting. This assistance occurred across three primary areas of the development workflow. First, Claude and ChatGPT aided in drafting and restructuring PROCRV sections, generating initial text for techni-

cal explanations, and converting content into LaTeX- or Markdown-compatible formats. All LLM-generated content was reviewed and edited by engineers to ensure accuracy, clarity, and alignment with internal documentation practices. Second, OpenAI Codex was employed to draft scaffolding for Lambda functions, AWS CDK constructs, error-handling snippets, and small utility functions. Developers validated, modified, or rewrote generated code to comply with Ricoh security and infrastructure requirements. Third, AMP and Claude Code helped interpret CloudFormation errors, identify IAM permission gaps, analyze Step Functions behavior, and provide debugging strategies throughout the deployment process.

The incorporation of LLMs reduced development time, improved documentation quality, and enhanced overall clarity of the system design without compromising security, governance, or operational safety.

Section 8 – Analysis of the Proposed System

8.1 Summary of Advantages

The agent significantly improves upon manual upstream monitoring. Detection latency reduces from weeks or months to a maximum of 6 hours via automated scheduled checking. Consistency improves through predictable update awareness regardless of team workload, detailed execution logs creating audit trails, and systematic tracking of upstream releases. Error elimination occurs through automated detection executing identical steps every 6 hours, ensuring reliable release identification and preventing missed updates. Post-merge deployment validation through smoke tests confirms successful updates before stakeholder use.

Sandbox currency improves through update awareness within 6 hours of release publication, enabling timely decisions about incorporating changes and reducing risk of outdated demonstrations. Additional benefits include reduced cognitive overhead, increased stakeholder confidence through notification-based visibility, enhanced developer productivity through automated validation, and improved Product team awareness via optional SNS subscriptions.

8.2 Phase 1 Scope and Implementation

The deployed Phase 1 system implements automated detection and notification with manual merge control, prioritizing infrastructure validation and security over immediate full automation.

Current capabilities include automated upstream release detection every 6 hours via EventBridge, SNS notifications with release details and GitHub links, manual fork synchronization using GitHub’s native interface, automated post-merge deployment triggering CloudFormation updates, workflow validation via Step Functions, and status reporting through SNS notifications and SSM Parameter Store updates.

The manual merge gate provides security by avoiding personal GitHub credentials, preserves quality control through human review, enables selective adoption of updates, and allows infrastructure validation before autonomous operation.

Phase 2 enhancement requires minimal effort. The PrepareMergeFn Lambda contains complete code for automated PR creation. Enabling this requires only creating a team GitHub service

account, generating a personal access token, storing it in SSM Parameter Store, and updating Lambda environment variables.

8.3 Proof of Concept Validation

A separate proof-of-concept exercise using Claude demonstrated that automated agents can successfully navigate complex upstream merges with substantial code changes (approximately 15,000 lines, using 55% of the model's context window). The exercise confirmed agents can identify conflicts, propose resolutions, and execute merges requiring extensive manual effort. However, current capabilities require significant human supervision to handle edge cases and validate intermediate steps.

This validation demonstrates agent-assisted updates are technically feasible rather than speculative. While fully autonomous operation remains a future goal, the hybrid model where agents accelerate routine tasks while humans handle exceptions delivers substantial value and confirms future automated merge handling is viable.

8.4 Summary of Disadvantages and Limitations

Phase 1 limitations constrain automation benefits. Manual fork sync means teams gain automatic awareness but must still perform manual GitHub operations. The six-hour detection cycle introduces latency between publication and notification, acceptable for sandbox environments but potentially unsuitable for time-critical scenarios.

Infrastructure dependency creates operational risk. The agent requires stable access to multiple AWS services, GitHub API, and SNS topics. Unlike manual processes allowing improvised workarounds, the automated agent cannot adapt to infrastructure instability. Mitigation requires robust monitoring, alerting, and documented fallback procedures.

Validation depth is limited. Current checks verify workflow stability and Step Functions execution but do not validate extraction accuracy or functional correctness, creating potential for updates to pass validation while degrading performance. This conscious scope decision prioritizes deployment stability, with future iterations potentially incorporating accuracy validation based on stakeholder feedback.

False test outcomes pose inherent risks: false positives from incomplete code path coverage or false negatives from timing issues or environmental variations. Mitigation requires continuous test refinement, escalation protocols, and periodic manual audits.

Training requirements introduce cost and temporary productivity impact through needed skills in SNS management, CloudWatch logs, Step Functions operation, and fork sync procedures. Comprehensive runbooks and progressive adoption mitigate but cannot eliminate this learning curve.

8.5 Alternatives and Trade-Offs Considered

Four alternatives were evaluated before selecting the Phase 1 approach.

The manual process (status quo) required no development effort but perpetuated all identified deficiencies: no systematic monitoring, unpredictable awareness latency, no audit trail, and no deployment validation. Rejected for addressing none of the core problems.

Immediate full automation (Phase 2 first) offered end-to-end optimization but required personal GitHub credentials during development, prevented infrastructure validation before autonomous operation, and increased implementation risk. Rejected in favor of phased deployment establishing infrastructure confidence first.

External CI/CD platforms (e.g., GitHub Actions) provided mature automation and rich integrations but introduced control limitations, credential exposure concerns, visibility constraints, and potential compliance issues. Rejected in favor of maintaining full operational control via internal AWS infrastructure.

The Phase 1 Lightweight Agent (selected) provides automated monitoring while preserving manual merge control, validates infrastructure before full automation, eliminates personal credential requirements, enables operational pattern establishment, and delivers immediate value through update awareness. Disadvantages include manual sync requirements, six-hour detection latency, initial setup effort, and ongoing maintenance responsibility. However, the manual gate is temporary and provides security benefits, latency is acceptable for sandbox environments, setup is one-time, and maintenance is manageable given focused scope.

This approach optimized for security through credential elimination, infrastructure validation through phased deployment, immediate value delivery despite manual merge preservation, and Phase 2 readiness through complete implementation requiring only configuration changes.